pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Hidden Layers and Activations

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

What a hidden layer adds

Inputs, a hidden layer, an output

19 inputs

the card columns, scaled on the fit rows

hidden units

each a neuron: a total, then an activation

1 output

day 2's sigmoid neuron, on the hidden outputs

Today

1. Three activations and their slopes
2. Two hidden units by hand, and in Excel
3. forward_pass and parameter_count
4. A ladder of six networks
5. One report on the validation rows

One neuron draws a straight line before the sigmoid. Activations let a network bend.

How an activation works

1. A straight total

z = b + w1 x1 + w2 x2 + ...

2. The bend

an activation of z

3. Three shapes

sigmoid 0 to 1, tanh -1 to 1, ReLU max(0, z)

4. The slope

where a curve is flat, a unit barely learns

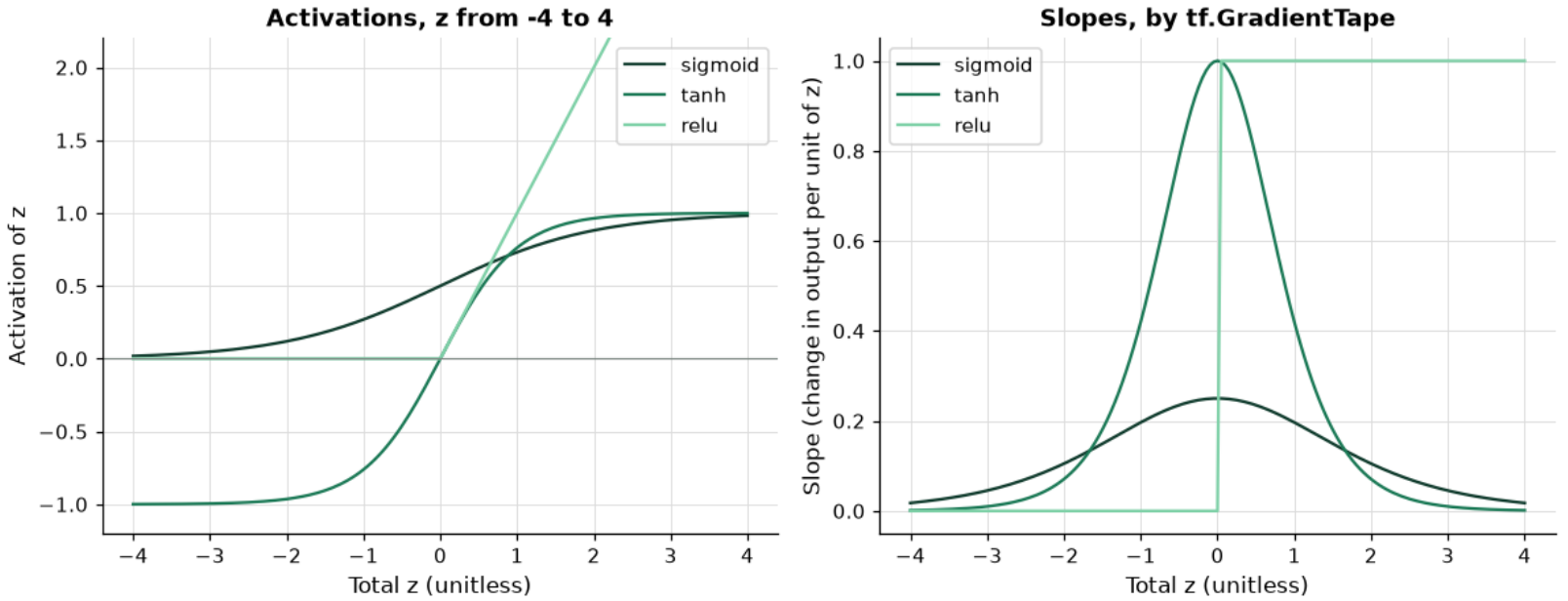
You control

The activation for each layer.

What comes out

One output per unit.

Activations as functions



0.2500

sigmoid's largest slope

1.0000

tanh's largest slope

1.0000

ReLU's slope above 0

How softmax works

1. One unit per group

each gives a score

2. Exponentiate

e to each score

3. Divide by the sum

probabilities that add to 1

4. Loss

minus the log of the true group's probability

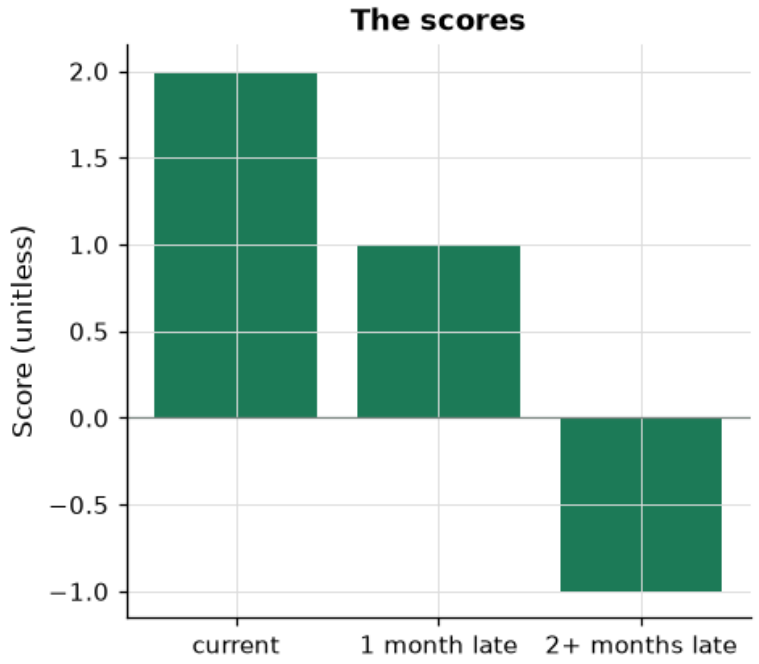
You control

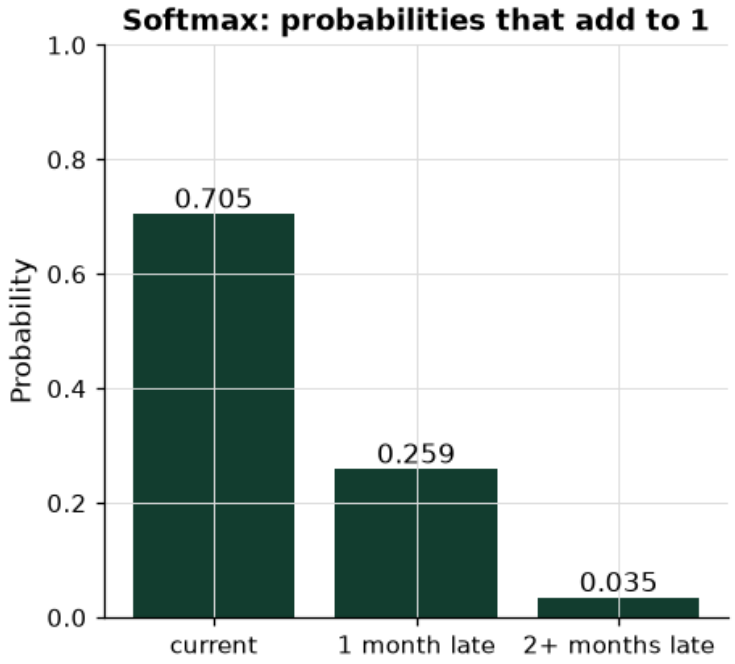
The number of groups.

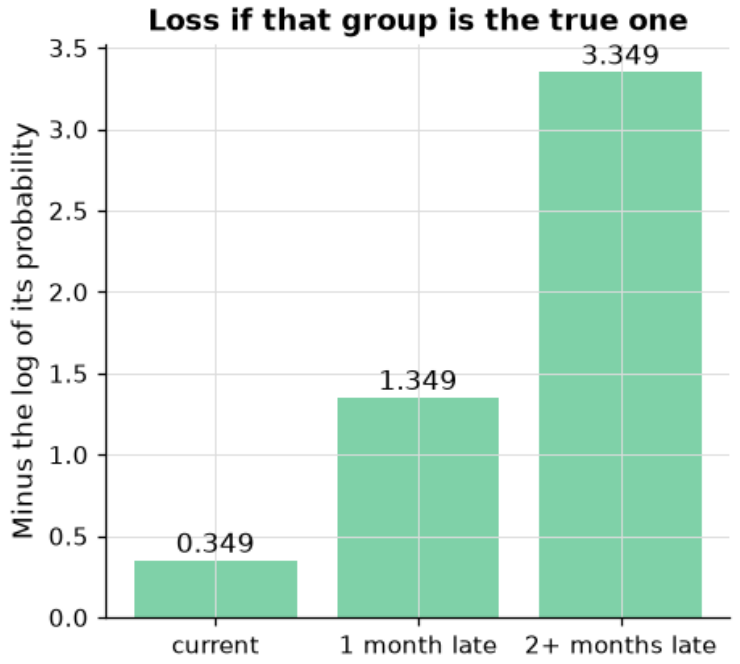
What comes out

One probability per group; with two groups it is the sigmoid.

Softmax: probabilities for more than two groups







Loss if the true group is the second: 1.3490. With two groups, softmax of (z, 0) is the sigmoid of z: 0.785835.

How a hidden layer works

1. One boundary

a neuron draws one straight line

2. A row of units

each its own weights and activation

3. Combine

the next layer reads their outputs

4. Bend

the boundary can bend

You control

The width, the activation, and the depth.

What comes out

One output per unit, passed to the next layer.

A hidden layer of two units, by hand

Weights (chosen by hand)	on PAY_0	on LIMIT_BAL	intercept
hidden unit 1	1.2	-0.3	-0.2
hidden unit 2	0.4	-0.9	0.1
output (on units 1, 2)	1.5	0.8	-1.6

First fit row

PAY_0 1.8078, limit -1.0596. ReLU unit 1 2.2873, unit 2 1.7768; output 0.9628 (tanh network 0.6513).

```
hidden = np.maximum(0.0, rows @ k + b)          # or np.tanh(rows @ k + b)
output = 1 / (1 + np.exp(-(hidden @ v + c)))     # equal to model.predict within 1e-6
```

Second row: unit 1 is 0.0000, a silent ReLU unit. 8,680 fit rows have at least one; a tanh unit is never silent.

How the forward pass works

1. Total

inputs times the kernel, plus the bias

2. Activation

applied to each unit's total

3. Pass on

one layer's output is the next one's input

4. Last layer

its output is the network's output

You control

Nothing: the weights and activations fix it.

What comes out

Every layer's output; the last is model.predict's.

forward_pass: the output layer by layer

```
for layer in model.layers:
    if Dropout: continue        # scoring time
    if not Dense: raise ValueError
    activation = layer.get_config()["activation"]
    kernel, bias = layer.get_weights()
    values = ACTIVATIONS[activation](values @ kernel + bias)
```

Three new tests

matches model.predict within 1e-5

passes Dropout through

refuses batch normalization

10

tests at the start

13

after forward_pass

15

after parameter_count

"linear" is what Keras stores for a Dense layer given no activation.

How parameters are counted

1. Weights

inputs x units

2. Intercepts

one per unit

3. Per layer

a x b + b

4. Sum

over the layers

You control

The widths and the depth; the count follows.

What comes out

One whole number: count_params() and by hand.

Counting parameters

```
a inputs into b units: a * b + b
```

Network	Sizes	Parameters
two units	2, 2, 1	9
Model 0	19, 1	20
Model 4	19, 32, 1	673
Model 5	19, 32, 16, 1	1,185

model.summary() of Model 5

Layer	Output shape	Param #
hidden_1	(None, 32)	640
hidden_2	(None, 16)	528
output	(None, 1)	17

```
Total params: 1,185
```

Shown once. Elsewhere: netkit.parameter_count and count_params(), one number each.

How the ladder works

1. Start

a starting model

2. One setting

change one setting per rung

3. Same way

fit on the fit rows, score on the stopping rows

4. Choose

by the stopping loss

You control

The starting model, the order of the rungs, and the rule for choosing.

What comes out

A table of models with one loss each, and a choice.

The ladder: one setting at a time

Model	Hidden layers	Parameters	Fit loss	Stopping loss
Model 0	one neuron	20	0.4617	0.4702
Model 1	16 sigmoid units	337	0.4380	0.4496
Model 2	16 tanh units	337	0.4253	0.4427
Model 3	16 ReLU units	337	0.4288	0.4449
Model 4	32 ReLU units	673	0.4256	0.4442
Model 5	32 and 16 ReLU units	1,185	0.4199	0.4448

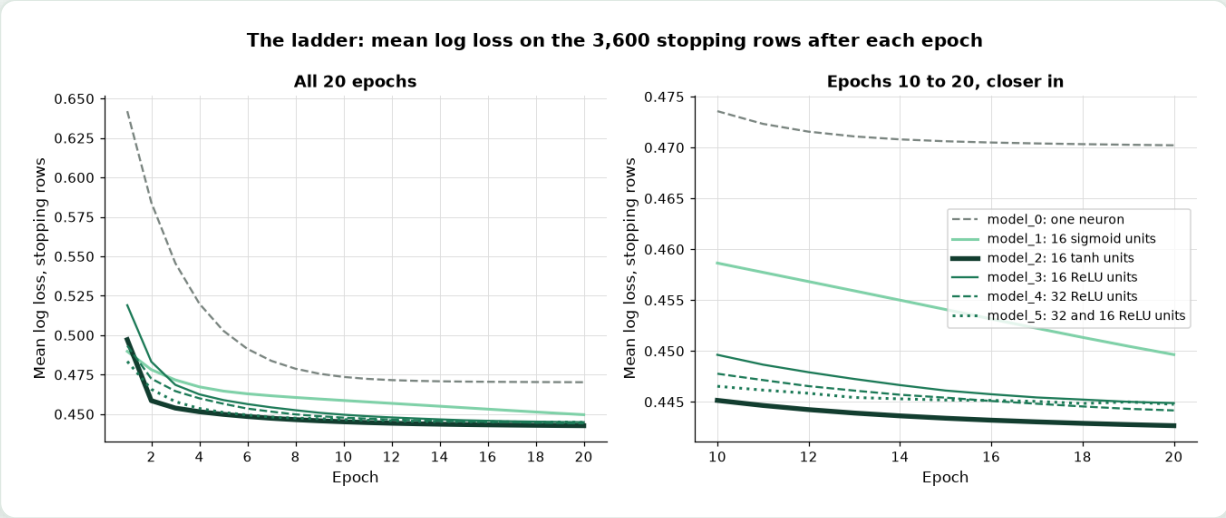
Chosen

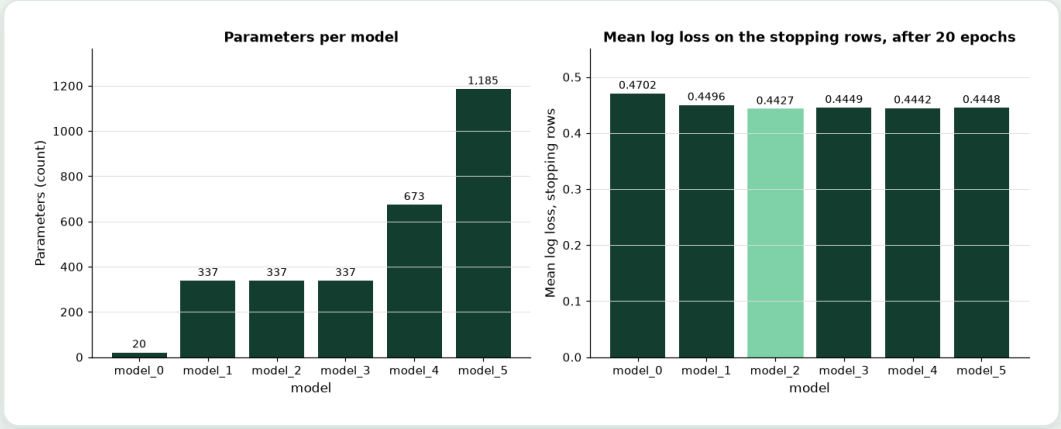
Model 2

lowest stopping loss among Models 1 to 5

20 epochs, batch 64, fitted on 14,400 rows; chosen on the 3,600 stopping rows. Models 2 to 5 lie within a seed's reach of each other.

The ladder, drawn





Every line lowest at epoch 20. Parameters from 20 to 1,185; stopping loss 0.4702 for Model 0, 0.4427 to 0.4449 for Models 2 to 5.

No activation, no hidden layer

```
keras.layers.Dense(32)                  # no activation= : a linear layer
w_one = k1 @ k2 @ k3                    # (19, 32) @ (32, 16) @ (16, 1) = (19, 1)
```

0.4702

linear stack, 1,185 parameters

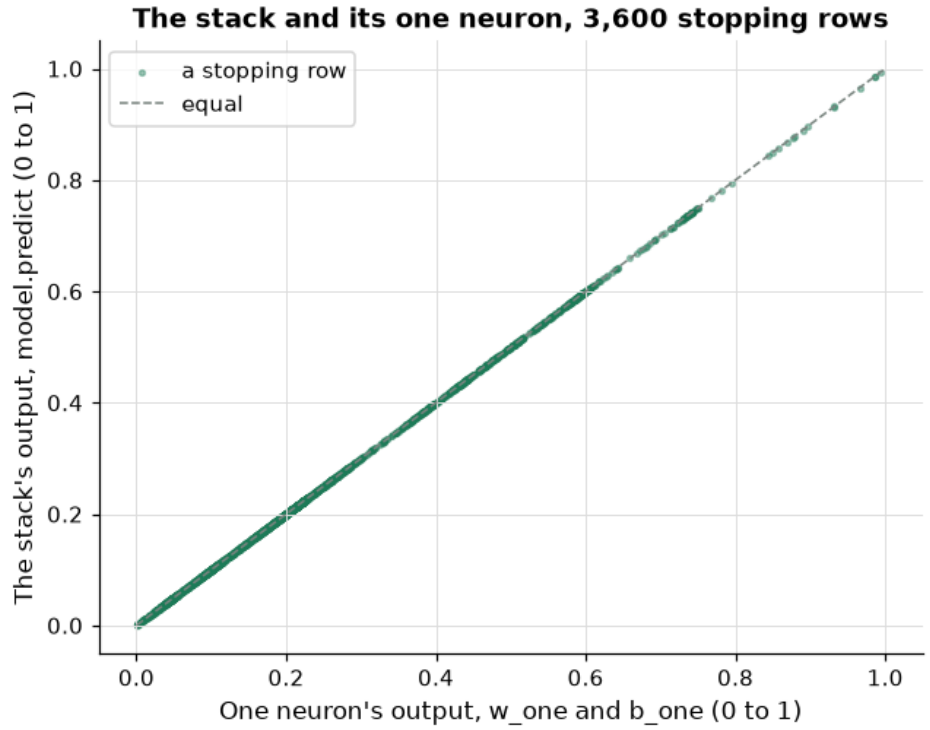
0.4702

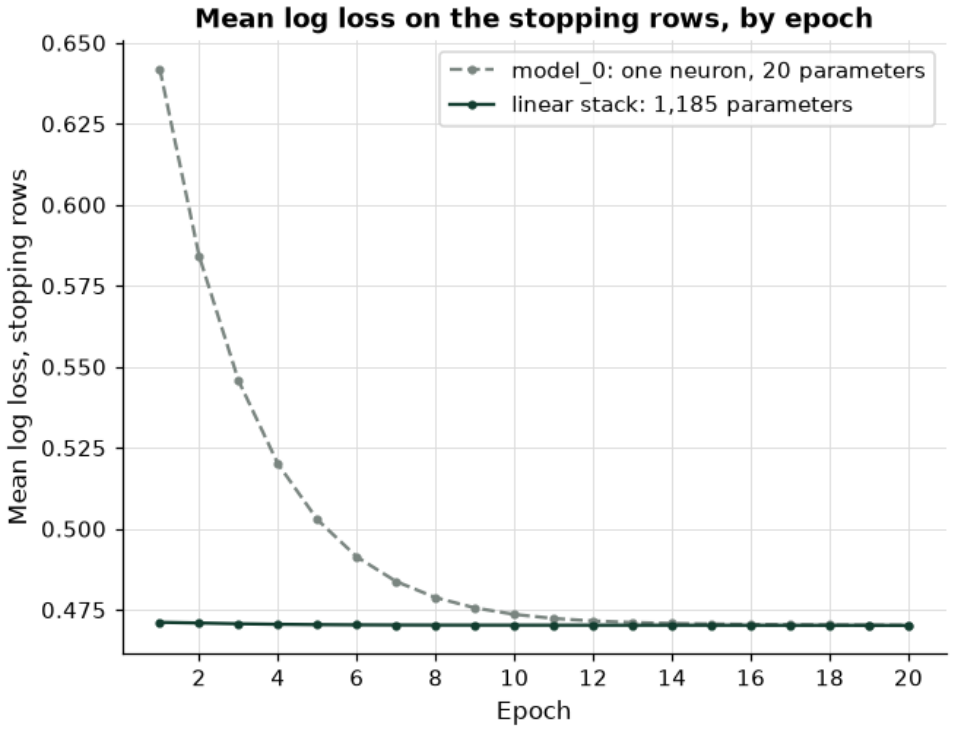
Model 0, one neuron, 20 parameters

One neuron with w_one and b_one equals the stack on every stopping row, within 1e-5.

The activation is what makes a hidden layer worth having. Keras does not warn you.

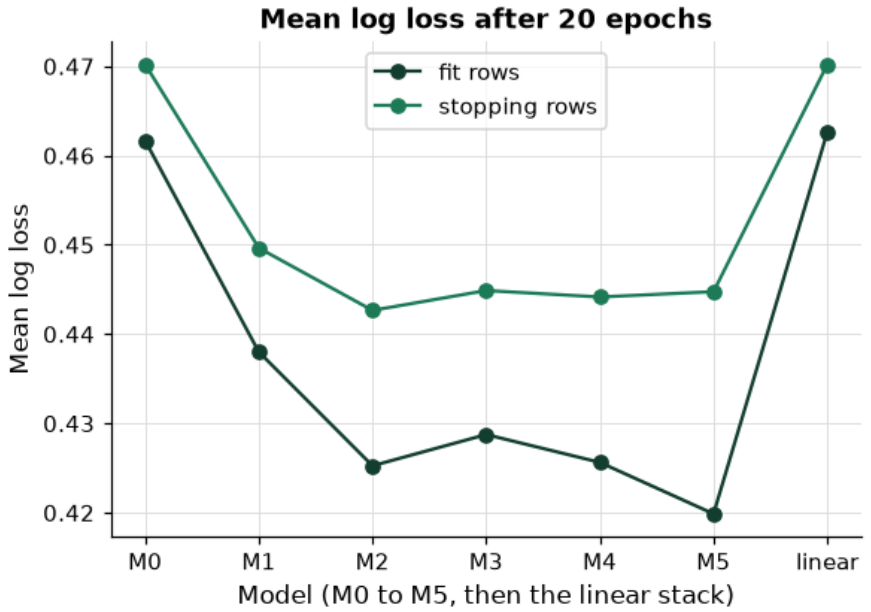
The stack, drawn

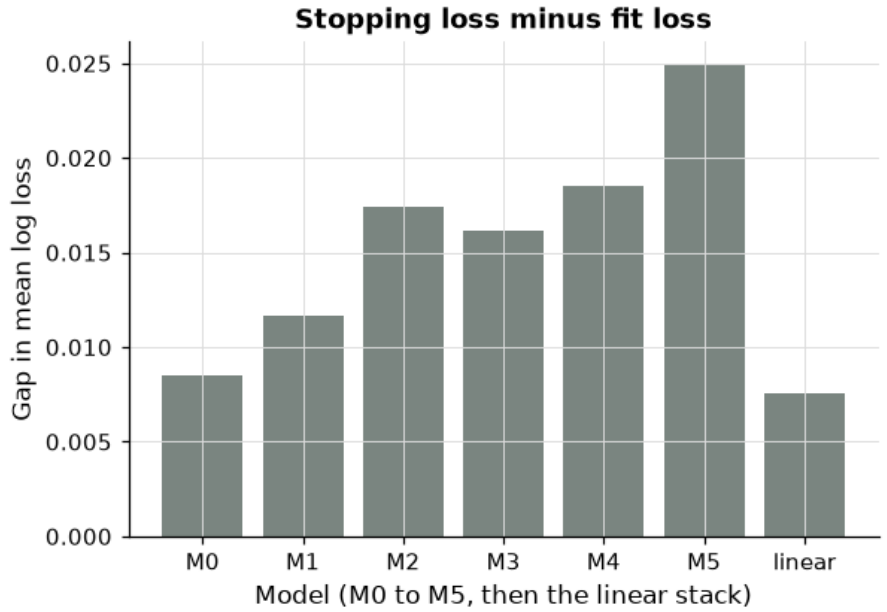




Every dot on the line. Both end at 0.4702: the stack got there faster, not further.

Width and depth on this file





Gap, stopping minus fit: 0.0161, 0.0185, 0.0249 for Models 3, 4, 5. Width helped a little here; depth did not.

The chosen network on the validation rows

0.7658

Model 2, validation ROC AUC

0.7168

Model 0, validation ROC AUC

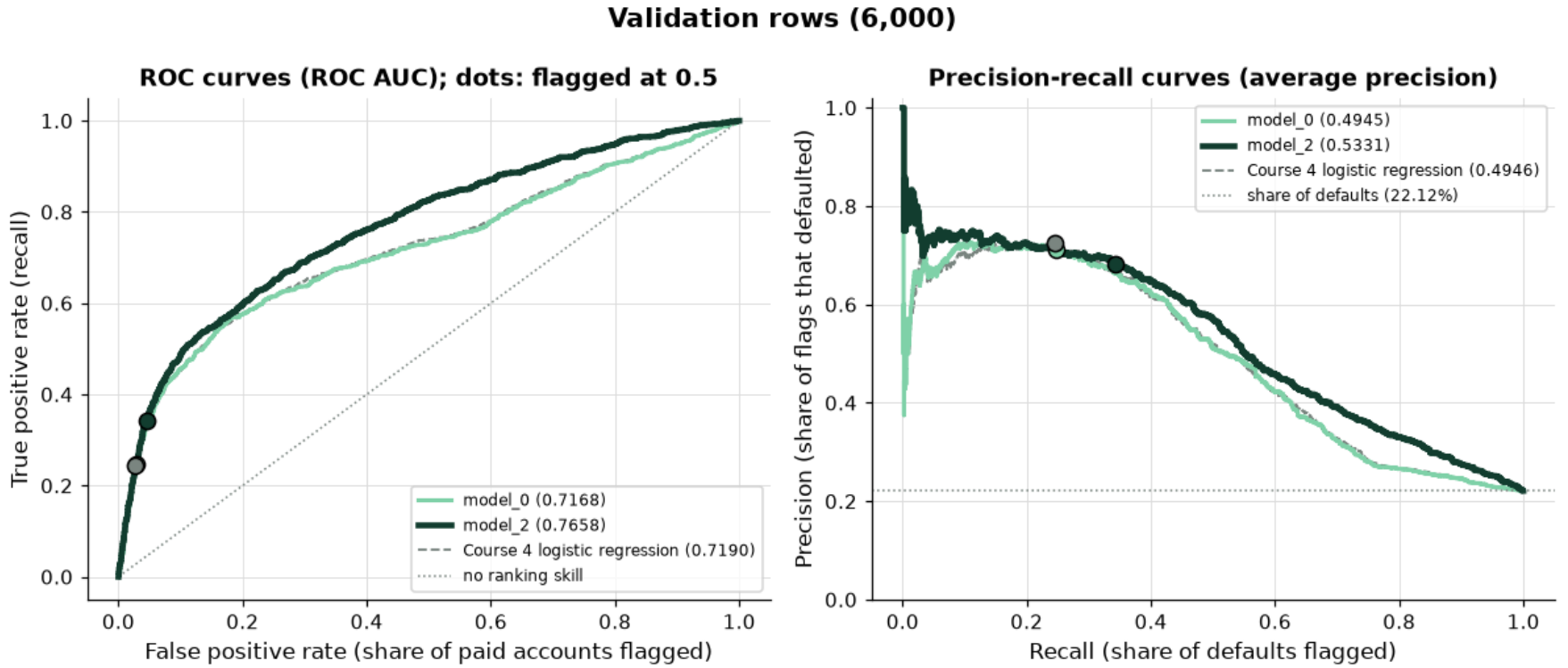
Average precision 0.5331 and 0.4945; Course 4 logistic regression ROC AUC 0.7190.

Day 5

Folds, a threshold, Course 5's LightGBM, and intervals before saying what the numbers support.

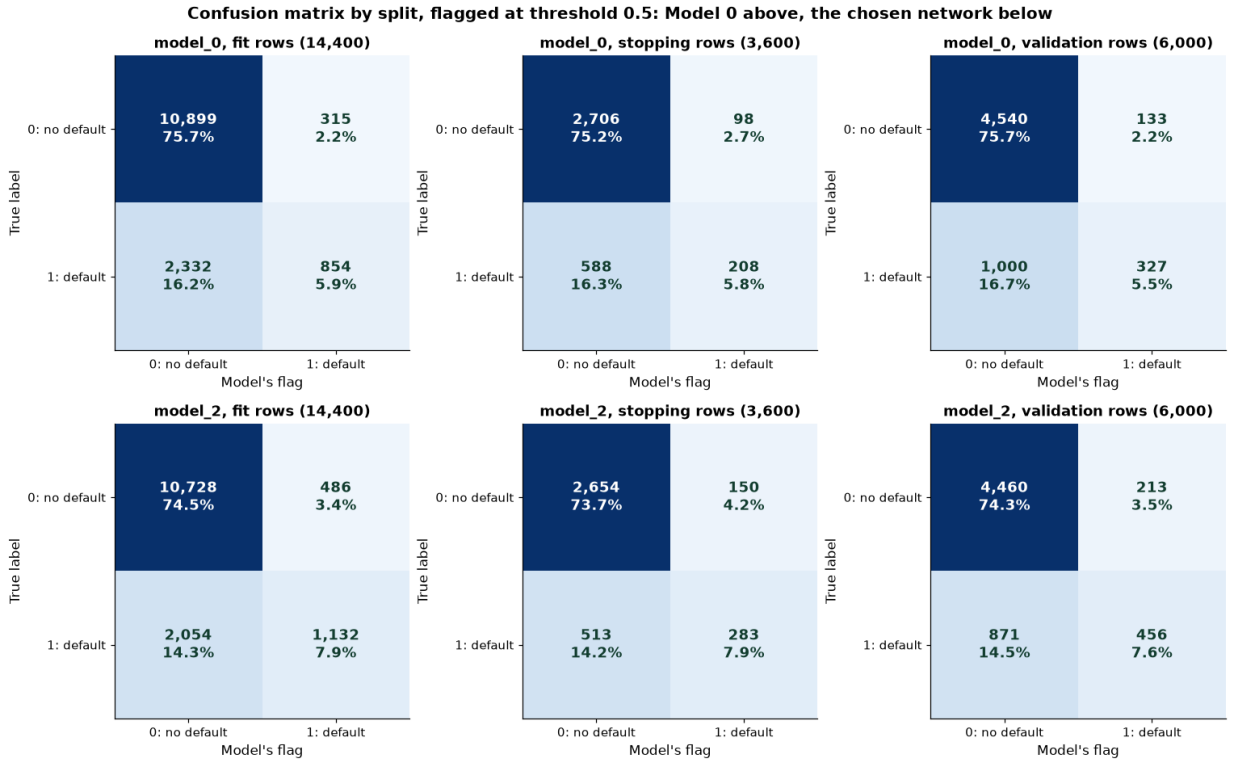
Chosen on the stopping rows; reported once on the validation rows.

Validation, drawn



ROC AUC: Model 2 0.7658, Model 0 0.7168, Course 4 0.7190. Model 0 and Course 4 are nearly one line.

Confusion matrices by split



Recall at 0.5, fit and validation rows: Model 0 26.8% and 24.6%; Model 2 35.5% and 34.4%.

An old name for one unit

F. Rosenblatt, 1958

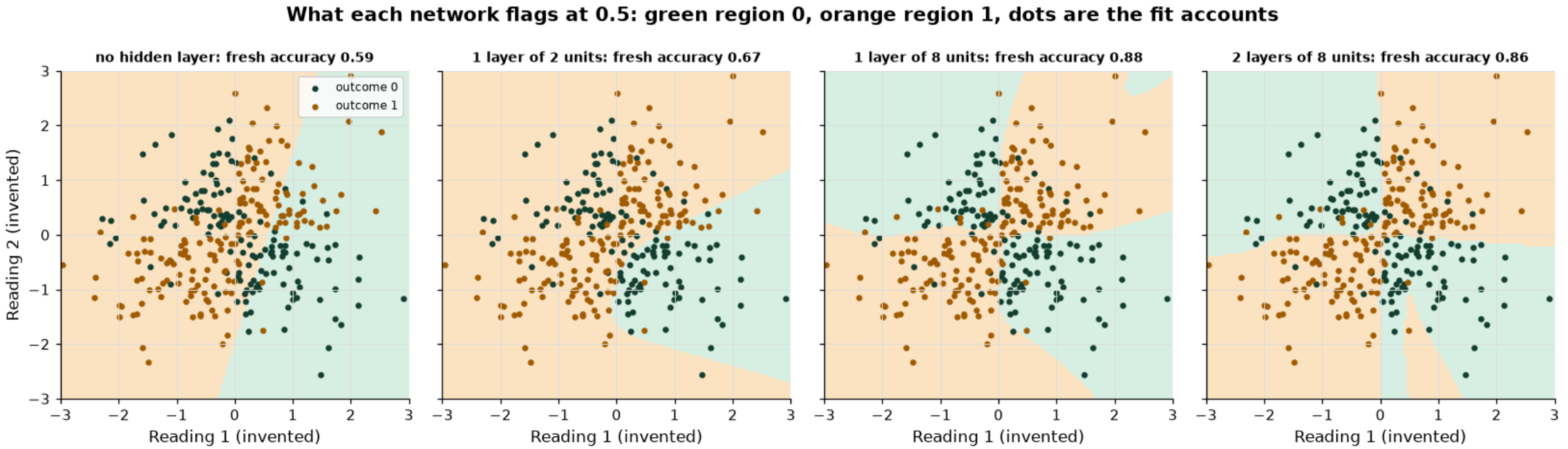
"The perceptron: A probabilistic model for information storage and organization in the brain."

Psychological Review, volume 65, issue 6, pages 386 to 408

A perceptron is one unit; a multilayer perceptron is units in layers, like Model 5.

Source, read 6 October 2026: the Crossref record of the paper (DOI 10.1037/h0042519). Bibliographic facts only; the paper itself was not read for this course.

What a hidden layer draws



Fresh accuracy: no hidden layer 0.590, 2 units 0.670, 8 units 0.883, two layers of 8 0.863. Invented pattern: no real file or market.

How a softmax network scores five buckets

1. One unit per bucket

five output units, each gives a score

2. Softmax

five probabilities that add to 1

3. The bucket

the one with the largest probability

4. The loss

minus the log of the true bucket's probability

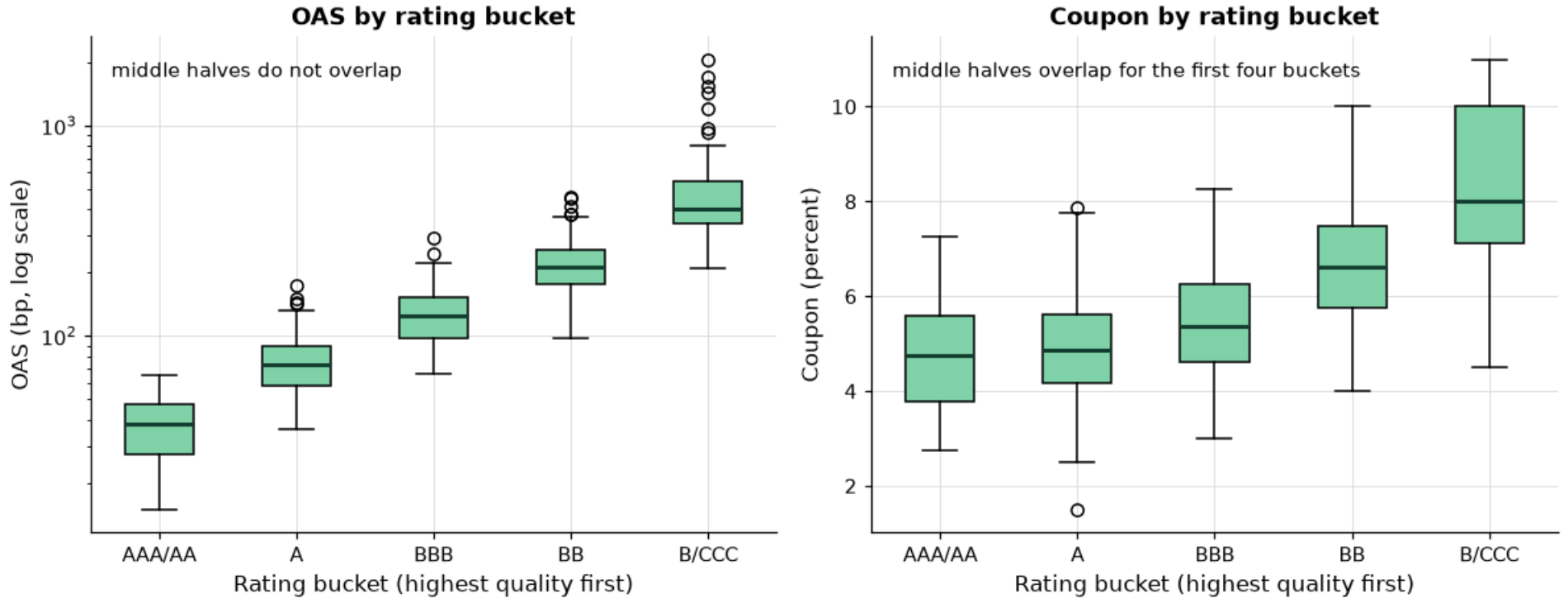
You control

The hidden layer. The number of buckets sets the five output units.

What comes out

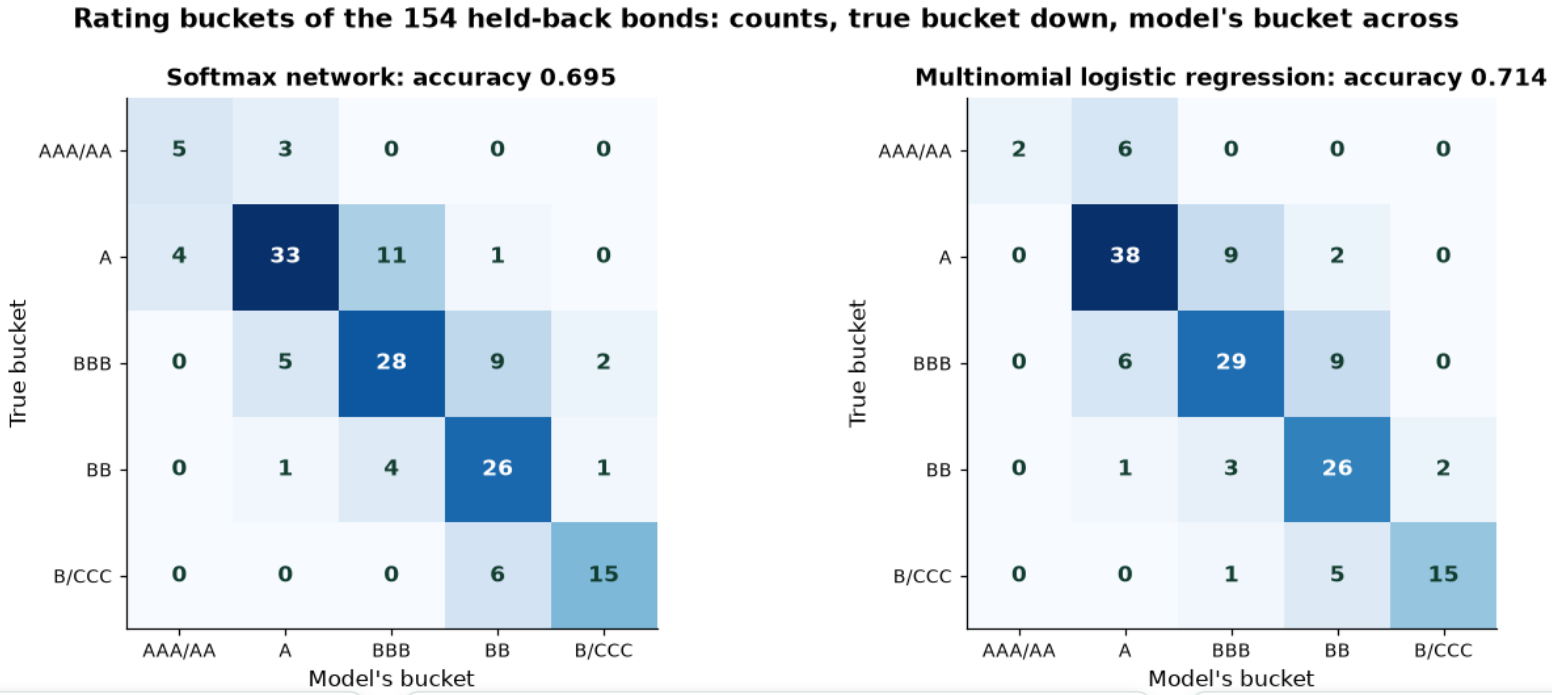
Five probabilities for every bond. A logistic regression is the same output with no hidden layer.

Five ordered rating buckets



Inputs: OAS, spread duration, and sector. No coupon: it reflects the rate regime at issue as much as credit quality. The bonds are synthetic and the generator set spreads by rating.

Confusion matrices for five buckets



0.6948

softmax network, accuracy

0.7143

logistic regression, accuracy

0.3182

always the largest bucket

How quadratic weighted kappa works

1. Observed

the confusion matrix

2. Expected

row total x column total / N

3. Weights

0 on the diagonal, squared distance, farthest pair 1

4. Kappa

1 - weighted observed / weighted expected

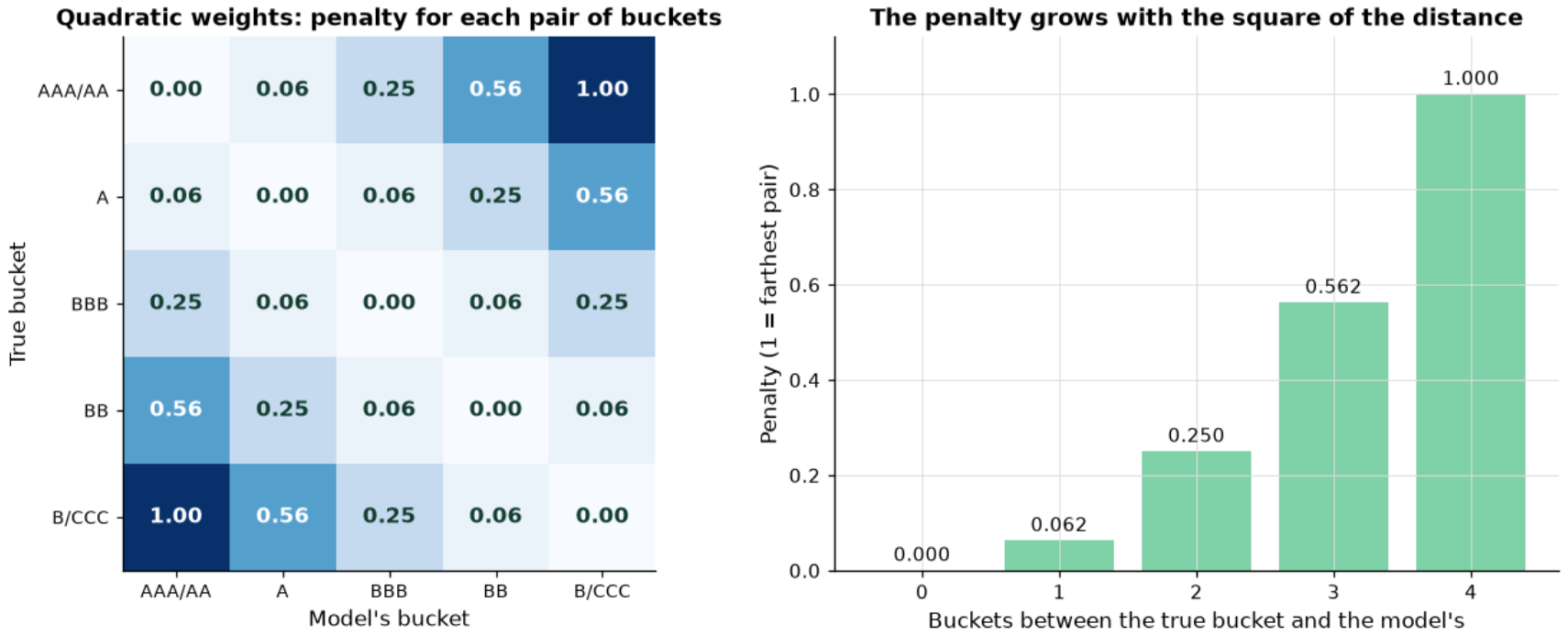
You control

The weights: quadratic here, linear in exercise 5.

What comes out

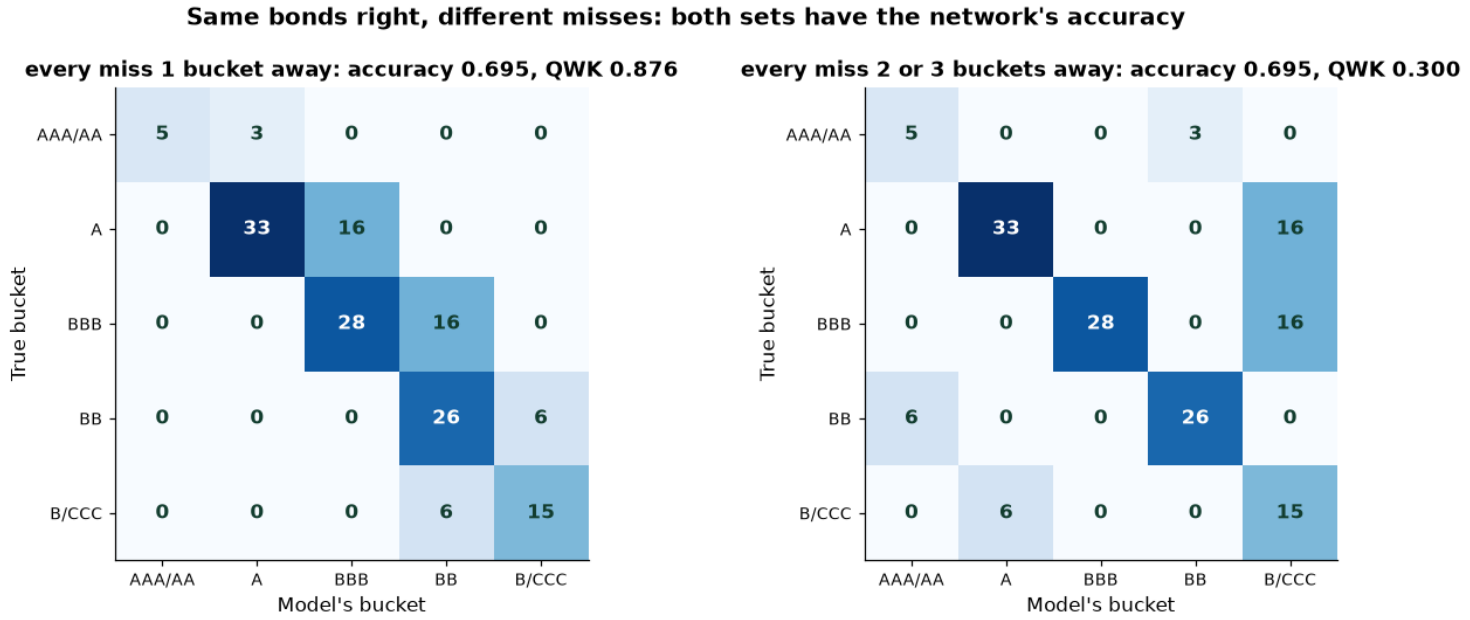
One number: 1 is perfect agreement, 0 no better than chance.

The weight table



By hand on the network: weighted disagreement 3.6875 observed against 24.1810 expected, so QWK 0.8475, equal to cohen_kappa_score within 1e-12.

Same accuracy, different kappa



Set of buckets	Bonds right	Accuracy	Largest miss	QWK
the network	107	0.6948	2	0.8475
every miss 1 bucket away	107	0.6948	1	0.8763
every miss 2 or 3 buckets away	107	0.6948	3	0.2996

Excel side by side

Excel side by side (tested in Excel)

`=MAX(0,A2)  =TANH(A2)  =1/(1+EXP(-A2))`	ReLU, tanh, sigmoid
`=MAX(0,SUMPRODUCT($A2:$B2,$L$2:$M$2)+$N$2)`	ReLU unit 1, in C2, filled down
`=1/(1+EXP(-(SUMPRODUCT(C2:D2,$L$5:$M$5)+$N$5)))`	the output neuron, in E2
`=TANH(SUMPRODUCT($A2:$B2,$L$2:$M$2)+$N$2)`	tanh unit 1, in F2
`=19*32+32+32*1+1`	Model 4: 673 parameters

No Keras or network training in Excel. On 14,400 fit rows: equal to forward_pass within 1e-12, to model.predict within 1e-6.

Quadratic weighted kappa in Excel

Excel side by side (tested in Excel)

`=COUNTIFS($A$2:$A$155,$G2,$B$2:$B$155,H$1)`	observed counts, over a 5 by 5 range
`=SUM($H2:$L2)*SUM(H$2:H$6)/SUM($H$2:$L$6)`	expected counts: row total x column total / N
`=($G18-H$17)^2/(5-1)^2`	quadratic weights
`=1-SUMPRODUCT(H18:L22,H2:L6)/SUMPRODUCT(H18:L22,H10:L14)`	QWK: 0.8475
`=ABS($G26-H$25)/(5-1)`	linear weights, for exercise 5

On the 154 held-back bonds: QWK 0.8475, linear kappa 0.7345, accuracy 0.6948, each equal to scikit-learn within 1e-12.

Save the day: two commits of the log

1. Code

mlkit, ensemblekit, netkit and its tests

2. Model 0

results.csv, first row

3. Model 2

the chosen network added

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
model_0_one_neuron,validation,0.7168,0.4945,0.5000,0.8112,0.7109,0.2464,0.3660
model_2_16_tanh,validation,0.7658,0.5331,0.5000,0.8193,0.6816,0.3436,0.4569
```

```
git log --oneline -- results.csv      # only the commits that changed the log
```

Threshold 0.5 until day 5. Accuracy beside the all-zero baseline, 0.7788, never to choose.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: hidden layers with no activation: 1,185 parameters, stopping loss 0.4702, one neuron's.

Code that runs is not the same as code that is right.

What you can do now

- Compute sigmoid, tanh, and ReLU; read their slopes
- Work a hidden layer by hand, in numpy and Excel
- forward_pass and parameter_count against Keras
- Climb a ladder; spot a Dense layer with no activation
- Fit a softmax network to five buckets; score it with QWK

Week 1, Day 4 Batch size, epochs, and loss curves